HOMEBITE

HOME-MADE FOOD MARKET PLACE PLATFORM

A MINI PROJECT REPORT

Submitted by

R. Ravi Kumar (B210071)

M. Rajendra Prasad (B211650)

K. Naveen (B210581)

Of

Bachelor of Technology

in Computer Science & Engineering

Under the Guidance of

Mrs. Devyani, Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies, Basar

Basar, Telangana

April 2026

HOMEBITE

HOME-MADE FOOD MARKET PLACE PLATFORM

Project Report submitted to Rajiv Gandhi University Of Knowledge Technnolgies, Basar for the partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in Computer Science & Engineering

by

R. Ravi Kumar (B210071)

M. Rajendra Prasad (B211650)

K. Naveen (B210581)

Under the Guidance of

Mrs.Devyani , Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies, Basar

Basar, Telangana

April 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies, Basar

CERTIFICATE

This is to certify that the project report entitled **HomeBite – Home-Made Food Marketplace Platform** submitted by **Ravi Kumar (B210071), Rajendra Prasad (B211650), and K Naveen (B210581)**, Department of Computer Science and Engineering, **Rajiv Gandhi University of Knowledge Technologies, Basar** for the partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering, is a bonafide record of the original work and investigations carried out by them under my supervision and guidance. The matter embodied in this report has not been submitted for the award of any other degree or diploma.

PROJECT SUPERVISOR	**HEAD OF DEPARTMENT**
Mrs. Devyani	**Dr.B.Venkata Ramana**
Assistant Professor	**Assistant Professor**

EXTERNAL EXAMINER

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies, Basar

DECLARATION

We hereby declare that the work presented in this project entitled, **"HomeBite – Home-Made Food Marketplace Platform"**, submitted to **Rajiv Gandhi University of Knowledge Technologies, Basar**, is an authentic record of our own work carried out under the supervision of **Mrs. Devyani**, **Assistant Professor,** Department of Computer Science and Engineering. The matter embodied in this project report has not been submitted by us for the award of any other degree or diploma to this university or any other institution.

Place: Basar

Date :

R. Ravi Kumar (B210071)

M. Rajendra Prasad (B211650)

K. Naveen (B210581)

ACKNOWLEDGEMENT

We would like to express our profound gratitude and sincere appreciation to all those who have contributed to the successful completion of this project. The development of HomeBite – Home-Made Food Marketplace Platform has been a challenging yet fulfilling academic journey, made possible through the collective encouragement and guidance of many individuals.

First and foremost, we would like to express our deepest gratitude to our project supervisor, **Mrs. Devyani, Assistant Professor**, Department of Computer Science and Engineering, Rajiv Gandhi University of Knowledge Technologies, Basar. Their invaluable guidance, consistent motivation, critical feedback, and willingness to share their expertise have been the cornerstone of this project. The time and effort invested by them in reviewing our work and suggesting improvements is sincerely acknowledged.

We extend our heartfelt thanks to **Dr.B.Venkata Ramana,** Head of the Department of Computer Science and Engineering, for providing an environment conducive to research and innovation, and for all the administrative support extended throughout the course of the project.

We are equally grateful to all the faculty members of the Department of Computer Science and Engineering for their support, academic insights, and encouragement during the entire course of this programme. Their teachings have equipped us with the technical foundation necessary to undertake this project.

Our sincere thanks also go to the university management for providing access to computing resources, laboratory infrastructure, and the necessary tools that made the implementation of HomeBite possible. The availability of a robust development environment was instrumental in bringing this project to fruition.

We acknowledge the contributions of the open-source community — particularly the maintainers of Node.js, Express.js, Prisma, React Native, and Expo — whose excellent documentation and community support significantly eased the development process.

Finally, we owe special thanks to our parents and families for their unconditional support, patience, and encouragement throughout our academic career. Their belief in our abilities has been a constant source of inspiration. We also thank our friends and classmates for their constructive suggestions and moral support during the project.

R. Ravi Kumar (B210071)

M. Rajendra Prasad (B211650)

K. Naveen (B210581)

ABSTRACT

HomeBite is a production-grade, full-stack mobile marketplace platform engineered to bridge the gap between skilled home cooks and local customers seeking authentic, nutritious, and affordable home-cooked food. The platform operates as a structured two-sided digital marketplace in which home cooks — referred to as vendors — can register, build a verified digital storefront, list their food offerings with comprehensive dietary and pricing details, and manage incoming orders, while customers can discover nearby home-cooked food options, place orders, make secure payments, and track deliveries — all from within a single, unified mobile application.

The existing food delivery ecosystem is dominated by large commercial aggregators such as Zomato and Swiggy, which exclusively cater to registered restaurants with formal commercial licences. This leaves millions of talented home cooks without a structured digital channel to monetise their culinary skills. Concurrently, a significant consumer segment — comprising students, working professionals, and individuals seeking home-style food — relies on informal and non-scalable channels such as social media groups and word-of-mouth networks to locate home-cooked food. HomeBite addresses this structural gap with a comprehensive, mobile-first solution.

The system architecture follows a three-tier client-server model. The backend is a stateless RESTful API built with Node.js and Express.js, persisting data in a MySQL 8.0 database managed through Prisma ORM, which serves as the single source of truth for schema definitions, migrations, and type-safe query building. The mobile frontend is a cross-platform application built with React Native using the Expo SDK 51 framework, employing file-based navigation through Expo Router v3.5 and lightweight state management via Zustand. This architecture enables a single JavaScript codebase to produce native applications for both Android and iOS platforms.

HomeBite supports three distinct user roles — Customer, Vendor, and Admin — each with dedicated navigation stacks, role-specific dashboards, and access-controlled API endpoints. Key platform capabilities include a gated vendor onboarding workflow with admin approval prior to public listing, a single-vendor cart enforcing order integrity, price-snapshotted order

items to maintain historical accuracy, an integrated payment processing pipeline via Razorpay with server-side HMAC-SHA256 signature verification, logistics tracking integration via Shiprocket, a post-delivery review and rating system restricted to completed and verified orders, and a comprehensive admin analytics panel for platform oversight.

Security is enforced through a stateless JWT dual-token authentication model with 15-minute access tokens and 7-day refresh tokens, incorporating rotation and reuse detection to mitigate token theft. Additional security measures include bcrypt password hashing with ten salt rounds, declarative request validation using Joi, HTTP security hardening through Helmet middleware, CORS whitelisting, and IP-based rate limiting at 200 requests per 15-minute window. The architecture supports horizontal scaling and is containerised with Docker and Docker Compose for production deployment through a CI/CD pipeline.

Comprehensive testing — encompassing unit, integration, system, performance, and security testing — validated all 28 functional requirements. The platform demonstrates acceptable performance under normal and medium load conditions, correctly enforces the order state machine, and maintains security against common web vulnerabilities including SQL injection, XSS, CSRF, and authentication bypass attempts. HomeBite conclusively demonstrates that a production-grade two-sided marketplace, covering authentication, multi-role access control, payment processing, logistics integration, and a fully functional mobile frontend, can be designed and implemented as a cohesive, well-architected system using modern open-source technologies.

Keywords: *Two-Sided Marketplace, Mobile Commerce, Node.js, React Native, JWT Authentication, Prisma ORM, RESTful API, Food Technology, Vendor Onboarding, Razorpay, Shiprocket, Docker.*

TABLE OF CONTENTS

Certificate iii

Declaration iv

Acknowledgement v

Abstract vi

Table of Contents vii

List of Figures ix

List of Tables x

List of Abbreviations and Symbols Used xi

1. Introduction 1

1.1 Project Overview 1

1.2 Purpose of the System 2

1.3 Scope of the Project 2

1.4 Motivation 3

1.5 Objectives 4

2. Literature Survey 5

2.1 Related Work 5

2.1.1 Existing Platforms 5

2.1.2 Research in Marketplace Technology 6

2.1.3 Gap Analysis 7

3. System Analysis 8

3.1 Problem Statement 8

3.2 Existing System Limitations 9

3.3 Proposed System 10

3.4 Functional Requirements 10

3.5 Non-Functional Requirements 13

3.6 Hardware and Software Requirements 14

4. System Design 15

4.1 Architecture 15
4.2 UML Diagrams 17
4.3 Data Flow Diagrams 19

5. Database Design 20
5.1 Database Overview 20
5.2 Table Definitions 20
5.3 Entity-Relationship Diagram 23
5.4 Key Design Decisions 24

6. Technology Stack and Implementation 25
6.1 Backend Technologies 25
6.2 Mobile Application Technologies 27
6.3 Project Structure 28
6.4 API Design 29

7. Security Design 32
7.1 Authentication Architecture 32
7.2 Role-Based Access Control 33
7.3 Payment Security 34
7.4 Additional Security Measures 34

8. Deployment Architecture 36
8.1 Production Environment 36
8.2 Docker Configuration 37
8.3 CI/CD Pipeline 37
8.4 Mobile App Distribution 38

9. Results and Testing 39
9.1 Testing Methodology 39
9.2 Test Cases 40
9.3 Performance Testing 41
9.4 Security Testing 41

9.5 Test Results Summary 42

10. Conclusion and Future Scope 43

10.1 Conclusion 43

10.2 Future Scope 44

References 46

LIST OF FIGURES

Figure 4.1 High-Level System Architecture Diagram 16

Figure 4.2 Module Interaction Flow Diagram 17

Figure 4.3 Use Case Diagram of HomeBite 17

Figure 4.4 Class Diagram of HomeBite 18

Figure 4.5 Sequence Diagram – Order Placement and Payment 18

Figure 4.6 Context Level Data Flow Diagram 19

Figure 4.7 Level 1 Data Flow Diagram 19

Figure 5.1 Entity-Relationship Diagram 23

Figure 7.1 JWT Dual-Token Authentication Flow Diagram 32

Figure 7.2 Order State Machine Diagram 33

Figure 8.1 CI/CD Deployment Pipeline 37

LIST OF TABLES

Table 3.1 Functional Requirements – User Registration and Authentication 11

Table 3.2 Functional Requirements – Vendor Onboarding 11

Table 3.3 Functional Requirements – Food Listing 12

Table 3.4 Functional Requirements – Cart and Order Management 12

Table 3.5 Hardware and Software Requirements 14

Table 4.1 External Service Integrations 16

Table 5.1 users Table – Column Definitions 20

Table 5.2 vendors Table – Column Definitions 21

Table 5.3 food_items Table – Column Definitions 21

Table 5.4 orders Table – Column Definitions 22

Table 5.5 order_items Table – Column Definitions (with Price Snapshotting) 22

Table 5.6 payments Table – Column Definitions 22

Table 5.7 deliveries Table – Column Definitions 23

Table 6.1 Backend Supporting Libraries 26

Table 6.2 Mobile Supporting Libraries 28

Table 7.1 RBAC Role Definitions 33

Table 7.2 Additional Security Measures 35

Table 8.1 Production Environment Components 36

Table 9.1 Sample Test Cases 40

Table 9.2 Performance Test Results 41

LIST OF ABBREVIATIONS AND SYMBOLS USED

API	Application Programming Interface
AWB	Air Waybill
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CUID	Collision-resistant Unique Identifier
DB	Database
FSSAI	Food Safety and Standards Authority of India
HMAC	Hash-based Message Authentication Code
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
MVP	Minimum Viable Product
ORM	Object-Relational Mapper
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SQL	Structured Query Language
UI/UX	User Interface / User Experience
UPI	Unified Payments Interface
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery

Chapter 1

INTRODUCTION

1.1 Project Overview

HomeBite is a full-stack, mobile-first marketplace platform engineered to connect home cooks — referred to as vendors — with local customers who seek authentic, nutritious, and affordable home-cooked food. The platform operates as a structured two-sided digital marketplace: home cooks register as vendors, build verified digital storefronts, list their food offerings complete with dietary and pricing information, and manage incoming orders; while customers browse vendor menus, place orders, make secure payments, and track deliveries — all through a single, unified cross-platform mobile application.

The system comprises two independently deployable technical components that work in concert. The first is a stateless RESTful backend API built with Node.js and Express.js, persisting relational data in a MySQL 8.0 database managed through Prisma ORM. The second is a cross-platform mobile application built with React Native using the Expo SDK 51 framework, enabling a single TypeScript codebase to produce fully native applications for both Android and iOS operating systems. Communication between the two components occurs exclusively through HTTPS REST API calls mediated by an Axios HTTP client.

The platform supports three distinct user roles — Customer, Vendor, and Administrator — each with dedicated navigation stacks, role-specific dashboards, and access-controlled API endpoints enforced through JWT-based Role-Based Access Control (RBAC). This multi-role architecture enables HomeBite to serve all participants of the home-cooked food ecosystem within a single, cohesive platform.

1.2 Purpose of the System

The core purpose of HomeBite is to create a trusted, structured, and scalable digital channel through which home-cooked food can be discovered, ordered, paid for, and delivered with full transparency. The system fulfils the following specific purposes:

- Provide home cooks with a verified digital storefront to list, manage, and monetise their culinary offerings without the barriers imposed by large commercial aggregators.

- Provide customers with a curated, browsable catalogue of home-made food vendors in their geographic area, complete with dietary information, pricing transparency, and reliable delivery tracking.
- Provide a platform administrator with comprehensive tools to approve vendors, manage user accounts, monitor platform activity, and access analytics dashboards.
- Integrate with established payment infrastructure (Razorpay) and logistics infrastructure (Shiprocket) so that vendors need not independently manage the complex technical and operational aspects of payment processing and delivery coordination.
- Enforce a verified onboarding process — including FSSAI licence capture and admin approval — to maintain food safety standards and build consumer trust in the platform.

1.3 Scope of the Project

The scope of the HomeBite project encompasses the design, implementation, testing, and documentation of the following functional domains:

- User Management: Secure registration, login, and profile management for three distinct roles — Customer, Vendor, and Admin — using JWT dual-token authentication with access token rotation.
- Vendor Management: A complete vendor onboarding and admin approval workflow encompassing business profile creation, FSSAI licence capture, geolocation recording, and an availability toggle for accepting orders.
- Food Catalogue: Creation, updating, soft deletion, and retrieval of food items with rich metadata including categories, images, pricing, discounted prices, preparation time, serving size, and dietary classification tags (VEG, NON_VEG, VEGAN, JAIN, GLUTEN_FREE).
- Cart and Ordering: A persistent single-vendor cart, order placement with price snapshotting, and a comprehensive order status lifecycle managed through a strictly enforced finite state machine.
- Payment Processing: Integration with Razorpay for payment order creation and server-side payment signature verification using HMAC-SHA256, preventing payment fabrication.

- Delivery Integration: Integration with Shiprocket for shipment creation, Air Waybill (AWB) assignment, and delivery tracking with live tracking URL provisioning.
- Review System: A customer review and rating mechanism restricted exclusively to orders with a DELIVERED status, ensuring review authenticity.
- Admin Panel: Vendor approval and rejection workflows, user management, platform-wide order visibility, and analytics.

The following features are explicitly out of scope for this version of the platform: a web-based customer storefront, direct real-time chat between customers and vendors, subscription or loyalty reward programmes, automated vendor payout disbursements, and OTP-based phone number verification.

1.4 Motivation

The Indian food delivery industry has experienced transformative growth over the past decade, with platforms such as Zomato and Swiggy revolutionising how urban consumers access restaurant food. However, these dominant platforms are almost exclusively designed for registered commercial restaurants with formal business licences, certified kitchen infrastructure, and established supply chains. This structural design leaves an enormous and underserved segment of the food economy entirely excluded from digital channels.

India is home to millions of talented home cooks — predominantly homemakers, retired individuals, culinary enthusiasts, and micro-entrepreneurs — who prepare high-quality food using fresh, locally sourced ingredients and traditional family recipes. These individuals currently have no structured, scalable, or professional means to offer their food to a wider audience beyond their immediate social networks. Their primary channels remain WhatsApp groups, social media posts, and word-of-mouth referrals — channels that are non-discoverable, non-scalable, cash-dependent, and devoid of order management, payment processing, delivery coordination, or dispute resolution mechanisms.

Simultaneously, a large and growing consumer segment — particularly college students, working professionals, elderly individuals living alone, and people relocating to new cities — actively seeks home-style, wholesome, affordable food that commercial restaurants cannot provide. These consumers currently resort to the same informal networks, leading to unreliable

ordering experiences, concerns about food safety, and a complete absence of the convenience features they expect from digital commerce.

HomeBite was conceived to address this structural gap with a purpose-built, technology-driven solution. By providing home cooks with professional digital tools and providing consumers with a reliable, transparent, and convenient ordering experience, HomeBite creates meaningful economic opportunity while simultaneously fulfilling a genuine consumer need. The application of modern software engineering principles — including layered architecture, secure authentication, price snapshotting, and integrated payment and logistics — transforms an informal, fragmented market into a structured, trustworthy digital ecosystem.

1.5 Objectives

The primary technical and functional objectives of the HomeBite project are:

1. To design and implement a modular, layered RESTful API architecture using Node.js and Express.js that cleanly separates routing, request validation, business logic, and data access concerns.
2. To develop a type-safe, migration-managed relational data layer using MySQL 8.0 and Prisma ORM, with a schema-first approach that ensures referential integrity, supports soft deletes, and maintains historical data accuracy through price snapshotting.
3. To implement a secure JWT dual-token authentication system with rotation and reuse detection, providing both security through short-lived access tokens and usability through long-lived silent refresh tokens.
4. To build a cross-platform mobile application using React Native and Expo SDK 51 with role-based navigation, automatic token refresh, persistent cart management, and clean separation between API services, state management, and UI rendering layers.
5. To integrate Razorpay's payment gateway with server-side HMAC-SHA256 payment verification, ensuring that no payment can be fabricated or bypassed on the client side.
6. To integrate Shiprocket's logistics API for automated shipment creation, AWB assignment, and real-time delivery tracking.

Chapter 2

LITERATURE SURVEY

2.1 Related Work

The development of HomeBite has been informed by a comprehensive survey of existing digital marketplace platforms, peer-reviewed research in marketplace technology and mobile commerce, and academic literature in software engineering and distributed systems design. This chapter reviews the most relevant prior work and identifies the gaps that HomeBite is specifically designed to address.

2.1.1 Existing Platforms

Zomato and Swiggy

Zomato and Swiggy represent the dominant food delivery aggregators in the Indian market. Both platforms provide sophisticated restaurant discovery, ordering, payment, and delivery tracking capabilities to millions of daily users. However, their onboarding requirements — which mandate formal FSSAI restaurant licences, commercial kitchen certifications, and established supply chains — create insurmountable barriers for home cooks operating at the micro-entrepreneur level. Neither platform provides any mechanism for individual home cooks to register as vendors and reach customers digitally. HomeBite is explicitly designed to fill this gap.

Airbnb Experiences

Airbnb's Experiences marketplace offers a conceptually parallel model to HomeBite's vendor onboarding design. Airbnb enables individual hosts to offer services — in this case, curated experiences — through a platform that provides them with a digital storefront, a payment processing layer, and a review system. The platform's trust architecture — built around identity verification, admin review of submissions, and a post-service review mechanism — directly informed HomeBite's vendor approval workflow and gated review system. The success of Airbnb Experiences demonstrates the commercial viability of a platform that empowers individual micro-entrepreneurs rather than institutional service providers.

Etsy

Etsy is a global marketplace for individual artisans and craftspeople, enabling them to sell handmade goods through professional digital storefronts with integrated payment processing and a review system. Etsy's model demonstrates how a platform can build consumer trust in a marketplace of individual creators — rather than established brands or retailers — through transparency, verified listings, and community reviews. HomeBite adopts a similar trust architecture, applying it to the food domain by requiring vendor verification before any listing becomes publicly visible.

Facebook Marketplace and WhatsApp Groups

Facebook Marketplace and community WhatsApp groups are the channels most commonly used by home cooks today to advertise and sell their food. These platforms offer zero friction for sellers but provide no payment integration, no delivery tracking, no order management, no dispute resolution, and no formal review mechanism. The dependency on cash payments and personal phone numbers for transactions creates significant risks for both buyers and sellers. HomeBite's design was directly informed by understanding the limitations of these existing channels and engineering solutions to each specific deficiency.

2.1.2 Research in Marketplace Technology

Two-Sided Marketplace Dynamics

Rochet and Tirole (2003) formulated the foundational theoretical framework for two-sided market platforms, demonstrating that such platforms must simultaneously address and balance supply-side (vendor) and demand-side (customer) growth to achieve network effects and market viability. Their analysis identified that premature vendor proliferation without quality control leads to consumer trust erosion and platform failure. HomeBite directly applies this insight through its admin-gated vendor approval workflow: vendors must be reviewed and approved before any listing becomes visible to customers, ensuring that quality is maintained even as the platform scales. The platform is therefore designed for controlled, quality-assured growth rather than unconstrained vendor registration.

Trust and Safety in Peer-to-Peer Platforms

Pavlou and Fygenson (2006) demonstrated through empirical research that trust mechanisms — including identity verification, transparent ratings, review systems, and dispute resolution processes — significantly increase purchase intent and reduce consumer anxiety in peer-to-peer commerce environments. Their research underscores the critical importance of reviews

and verification in markets where consumers transact with individuals rather than established brands. HomeBite implements this framework through its gated review system, which restricts the submission of reviews exclusively to customers who have completed verified DELIVERED orders, ensuring that all ratings reflect genuine transactional experiences rather than fabricated or unverified feedback.

Mobile Commerce Adoption

Yeh, Li, and Mahatanankoon (2020) conducted a comprehensive analysis of mobile commerce adoption factors, identifying that single-click ordering flows, integrated payment processes, persistent cart states, and push notification capabilities are the most significant drivers of mobile conversion rates and cart abandonment reduction. Their research validated the design choices made in HomeBite's mobile application — specifically, the persistent Zustand-managed cart that survives app session boundaries, the in-app Razorpay checkout that eliminates redirect-based payment abandonment, and the planned push notification capability for order status updates.

Microservices and Modular API Architecture

Newman (2015) and Richardson (2018) both advocate for modular, feature-based API architecture in which each domain of business logic is encapsulated within its own module containing routes, controllers, and services. This approach, which HomeBite implements across its auth, vendor, food-items, cart, orders, payments, delivery, reviews, and admin modules, enables independent evolution of business domains, simplifies testing by isolating concerns, and supports horizontal scaling of individual services as load demands evolve.

2.1.3 Gap Analysis

Based on the review of existing platforms and academic research, the following gaps have been identified that HomeBite specifically addresses:

- No existing platform provides an end-to-end structured marketplace for home cooks that combines FSSAI licence capture, admin verification, food safety transparency, integrated payments, and logistics coordination in a single mobile application.
- Existing informal channels such as WhatsApp groups and social media posts lack any form of payment integration, delivery tracking, order management, or dispute resolution, creating poor and unreliable experiences for both buyers and sellers.

- Large commercial aggregators such as Zomato and Swiggy do not support micro-vendor onboarding without formal commercial licensing, effectively excluding the home-cooked food segment from digital commerce.
- No solution currently combines vendor discovery, verified onboarding, ordering, payment processing, logistics integration, and gated reviews in a single cross-platform mobile application targeting the home-cooked food segment.
- Existing research confirms strong consumer demand for home-style food and mobile-first ordering, but no production-grade technical implementation has been documented that addresses this market with the security, scalability, and feature completeness that HomeBite delivers.

Chapter 3

SYSTEM ANALYSIS

3.1 Problem Statement

The home-cooked food market in India — and across much of the developing world — suffers from a fundamental structural problem: the supply side (skilled home cooks with high-quality food offerings) and the demand side (consumers seeking home-style, affordable food) exist in close geographic proximity but have no reliable, structured, or scalable mechanism to connect with each other. This disconnection creates the following specific, well-defined problems:

- Lack of Digital Storefront: Home cooks have no professional means to advertise their food offerings beyond their immediate social circle, limiting their potential customer base and economic opportunity.
- No Order Management System: There is no systematic mechanism for home cooks to receive, acknowledge, prioritise, and track incoming orders, leading to confusion, missed orders, and poor customer experience.
- Cash Dependency and Security Risk: Transactions depend on cash payments or informal personal UPI transfers, creating security risks, making accounting impossible, and deterring customers accustomed to digital payment.
- Absence of Discovery Mechanism: Customers have no reliable, location-aware means to discover nearby home-cooked food options, making the discovery process time-consuming and unreliable.
- Dietary Transparency Deficit: Customers lack access to verified dietary information — including allergen details, ingredients, and preparation methods — creating food safety concerns for individuals with dietary restrictions.
- No Dispute Resolution: In the absence of a formal platform intermediary, there is no standardised process for resolving disputes arising from order errors, quality issues, or non-delivery.
- No Analytics or Business Intelligence: Home cooks cannot access any data about their sales, customer preferences, revenue trends, or operational performance, preventing them from making informed business decisions.

3.2 Existing System Limitations

The traditional methods currently employed by home cooks to operate their food businesses exhibit the following critical limitations:

- WhatsApp Groups and Social Media: While these channels provide basic visibility within closed social networks, they offer no order management functionality, no payment processing, no delivery coordination, and no formal review mechanism. They are entirely dependent on personal relationships and manual communication.
- Manual Cash Collection: Cash-based or informal UPI-based transactions lack payment guarantees, create accounting complexity, expose both parties to fraud risk, and deter the growing segment of consumers who exclusively use digital payment methods.
- Verbal Delivery Coordination: Delivery arrangements depend on informal verbal agreements between cooks and customers, with no tracking, no delivery partner integration, and no accountability if deliveries fail.
- No Analytics: Home cooks have no access to order history, revenue data, or customer behaviour analytics, preventing any form of evidence-based business decision-making.
- Non-Scalability: The entire existing model is bounded by the cook's personal social network and ability to manage manual communications, making it fundamentally non-scalable.

3.3 Proposed System

HomeBite addresses all identified gaps by providing a comprehensive, structured, mobile-first digital marketplace that includes the following capabilities:

- A verified vendor onboarding process with administrator approval before any listing becomes publicly visible, maintaining quality and food safety standards.
- A structured, searchable, and filterable food catalogue with dietary tags, images, pricing, and preparation time information for complete consumer transparency.
- An integrated cart and ordering flow with single-vendor enforcement and price snapshotting at the time of order placement.
- Secure, in-app payment processing through Razorpay with server-side payment signature verification, eliminating cash dependency.

- End-to-end delivery tracking through Shiprocket integration, providing customers with live tracking information.
- A gated review system restricted to completed, delivered orders, ensuring review authenticity and building platform trust.
- An admin dashboard for comprehensive platform oversight, vendor management, and analytics.
- Role-based mobile navigation with dedicated interfaces for Customer, Vendor, and Administrator roles.

3.4 Functional Requirements

3.4.1 User Registration and Authentication

Table 3.1 presents the functional requirements for user registration and authentication.

FR-ID	Requirement Description
FR-01	A user shall register with name, email, phone, password, and role (CUSTOMER or VENDOR).
FR-02	The system shall prevent duplicate registrations using the same email address.
FR-03	Passwords shall be hashed using bcrypt with ten salt rounds before storage.
FR-04	On successful login, the system shall issue a JWT access token (15-minute expiry) and a JWT refresh token (7-day expiry).
FR-05	All authenticated API requests shall require a valid access token via the Authorization: Bearer header.
FR-06	When the access token expires, the client shall obtain a new pair using the refresh token. The old refresh token shall be invalidated immediately (token rotation).
FR-07	Reuse of a previously invalidated refresh token shall be rejected as a token-theft indicator.
FR-08	An authenticated user shall be able to change their password by providing their current and new passwords.
FR-09	An administrator shall be able to block a user account, preventing all future logins for that account.

3.4.2 Vendor Onboarding

Table 3.2 presents the functional requirements for vendor onboarding and management.

FR-ID	Requirement Description
FR-10	A VENDOR user shall submit an application with: business name, FSSAI number, city, state, pincode, logo image, and bank/UPI details.
FR-11	On submission, vendor status shall be set to PENDING. The vendor shall not appear in public listings until explicitly approved by an administrator.
FR-12	An administrator shall be able to approve, reject (with reason), or suspend a vendor application.
FR-13	An approved vendor shall be able to toggle their isAcceptingOrders flag to temporarily pause the receipt of new customer orders.
FR-14	Vendor profiles shall store latitude and longitude coordinates to enable proximity-based vendor filtering for customers.

3.4.3 Food Listing

Table 3.3 presents the functional requirements for food catalogue management.

FR-ID	Requirement Description
FR-15	An approved vendor shall create food items with: name, description, category, price, discounted price, preparation time, serving size, dietary tag, and image.
FR-16	Dietary tags shall be one of: VEG, NON_VEG, VEGAN, JAIN, GLUTEN_FREE.
FR-17	A vendor shall be able to toggle individual food item availability without permanently deleting the item.
FR-18	Food item deletion shall be a soft delete (isDeleted = true), preserving referential integrity with historical order items.
FR-19	Customers shall be able to browse and filter food items by category, dietary tag, or vendor.

3.4.4 Cart and Order Management

Table 3.4 presents the functional requirements for cart management and order processing.

FR-ID	Requirement Description
FR-20	A customer shall have exactly one active cart at

	any point in time.
FR-21	A cart shall be restricted to items from a single vendor. Any attempt to add items from a second vendor shall return an error.
FR-22	A customer shall be able to add items to, update item quantities in, and remove items from their active cart.
FR-23	Order placement shall snapshot item names and prices at the time of ordering. Future price changes by the vendor shall not affect historical order records.
FR-24	Upon successful order placement, the customer's cart shall be automatically cleared.
FR-25	Order status shall progress through: PENDING_PAYMENT → CONFIRMED → ACCEPTED → PREPARING → READY_FOR_PICKUP → OUT_FOR_DELIVERY → DELIVERED.
FR-26	A customer may cancel an order only when its status is PENDING_PAYMENT or CONFIRMED.
FR-27	A vendor may accept or reject a CONFIRMED order. Rejection requires a reason to be provided.
FR-28	A vendor shall update order status from ACCEPTED through READY_FOR_PICKUP as food preparation progresses.

3.5 Non-Functional Requirements

The following non-functional requirements govern the system's quality attributes:

- Performance: All list and catalogue endpoints shall support cursor-based or page-based pagination. Prisma queries shall use selective field inclusion (select and include) to retrieve only required fields, minimising data transfer overhead.
- Security: Password storage shall use bcrypt with ten salt rounds. Access tokens shall have a 15-minute expiry. Refresh tokens shall employ rotation with reuse detection. Rate limiting shall be applied at 200 requests per IP address per 15-minute window. All HTTP security headers shall be set through Helmet middleware. CORS shall be restricted to an explicit whitelist of allowed origins. All input shall be validated through declarative Joi schemas.
- Scalability: The backend shall be stateless, enabling horizontal scaling behind a load balancer without session affinity requirements. Prisma shall manage a configurable database connection pool to handle concurrent requests efficiently.

- Reliability: The server shall handle SIGTERM and SIGINT signals with a graceful shutdown sequence. All asynchronous controller functions shall be wrapped in an asyncHandler to ensure unhandled promise rejections are caught and processed by the global error handler.
- Compatibility: The React Native / Expo mobile application shall target both Android (API level 26+) and iOS (iOS 13+) platforms from a single codebase, providing a native user experience on both platforms.
- Maintainability: The backend shall follow a modular, feature-based directory structure. Each domain module shall encapsulate its routes, controller, and service files independently, enabling future modifications and additions without cross-module impact.
- Availability: The production system shall target a minimum uptime of 99.5% through containerisation, automated health checks, and process management via PM2.

3.6 Hardware and Software Requirements

Table 3.5 summarises the hardware and software requirements for the development and deployment of HomeBite.

Component	Specification
Processor	Intel i5 8th Gen or equivalent / higher
RAM	Minimum 8 GB (16 GB recommended)
Storage	Minimum 256 GB SSD
OS (Development)	Windows 10/11 / Ubuntu Linux / macOS
Backend Runtime	Node.js 18 LTS or higher
Database	MySQL 8.0
ORM	Prisma ORM v5.x
Mobile Framework	React Native with Expo SDK 51
Version Control	Git / GitHub
API Testing	Postman
Containerisation	Docker / Docker Compose
IDE	Visual Studio Code with ESLint and Prettier plugins

Chapter 4

SYSTEM DESIGN

4.1 Architecture

HomeBite follows a classical three-tier client-server architecture with a clear and enforceable separation between the presentation layer, the application logic layer, and the data storage layer. This architectural choice provides several critical benefits: it enables independent evolution and scaling of each tier, simplifies testing by isolating concerns, and allows the mobile frontend and the backend API to be deployed, versioned, and scaled independently.

4.1.1 Presentation Layer – React Native Mobile Application

The presentation layer is a cross-platform mobile application built with React Native using the Expo SDK 51 framework. Expo Router v3.5 provides file-based, type-safe navigation with automatic deep linking support, enabling the three distinct role-based navigation stacks — (customer), (vendor), and (admin) — to coexist within a single application. State management is handled through lightweight Zustand stores: the authStore manages user session state and access token lifecycle, while the cartStore manages cart items, computed totals, and mutation actions.

The application communicates with the backend exclusively through HTTPS REST API calls via an Axios HTTP client configured with request and response interceptors. The interceptors transparently inject the Authorization: Bearer access token header on all outbound requests and automatically handle token refresh on 401 Unauthorized responses, providing a seamless authentication experience for users. Access tokens are stored in Zustand memory only and are never persisted to device storage, while refresh tokens are persisted to device secure storage via expo-secure-store, which leverages the native Android Keystore and iOS Keychain mechanisms.

4.1.2 Application Layer – Node.js + Express.js REST API

The application layer is a stateless RESTful API implemented in Node.js with the Express.js framework, organised into feature-based domain modules. The architecture enforces a strict three-component pattern within each module:

- Routes: Each route file declares API endpoints and applies an ordered middleware chain consisting of authentication verification, role-based access control, and request body validation before passing control to the controller.
- Controllers: Controller functions are thin wrappers that invoke the corresponding service function and return a standardised JSON response envelope ({ success, message, data }) to the client. All controller functions are wrapped in an asyncHandler utility that eliminates the need for try-catch blocks and ensures all unhandled promise rejections are forwarded to the global error handling middleware.
- Services: Service files contain all business logic — including database queries through the Prisma client, third-party API calls, and domain-specific validation — entirely decoupled from the HTTP layer, facilitating independent unit testing.

4.1.3 Data Layer – MySQL + Prisma ORM

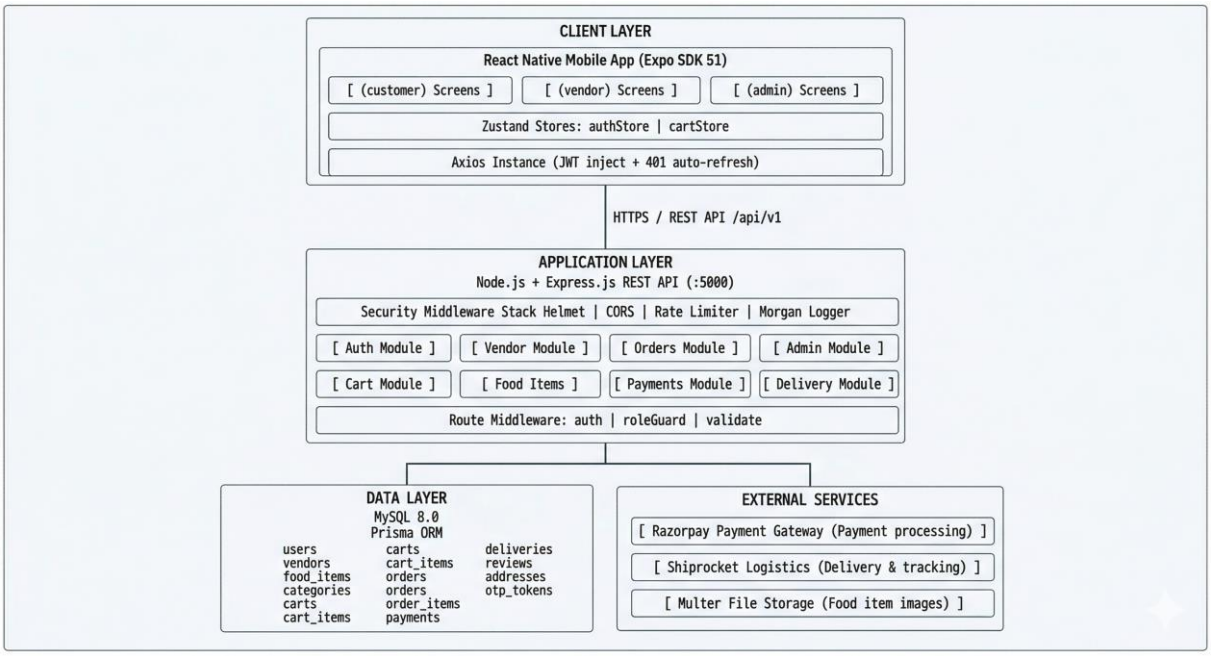
The data layer uses MySQL 8.0 as the relational database management system. All database access is mediated exclusively through Prisma ORM, which provides: a schema-first approach where the prisma/schema.prisma file serves as the single authoritative source of truth for all table definitions, column types, enumerations, and relational constraints; type-safe query building through the generated Prisma client with full TypeScript autocomplete support; and automatic, versioned migration management through prisma migrate. This approach eliminates the risk of schema drift between the database and the application code.

4.1.4 External Services

Table 4.1 summarises the external service integrations used by HomeBite.

Service	Purpose	Status
Razorpay	Payment order creation and HMAC-SHA256 signature verification	Stubbed (live-compatible)
Shiprocket	Shipment creation, AWB assignment, and delivery tracking	Stubbed (live-compatible)
Multer (local)	Food item image upload and static file serving	Active

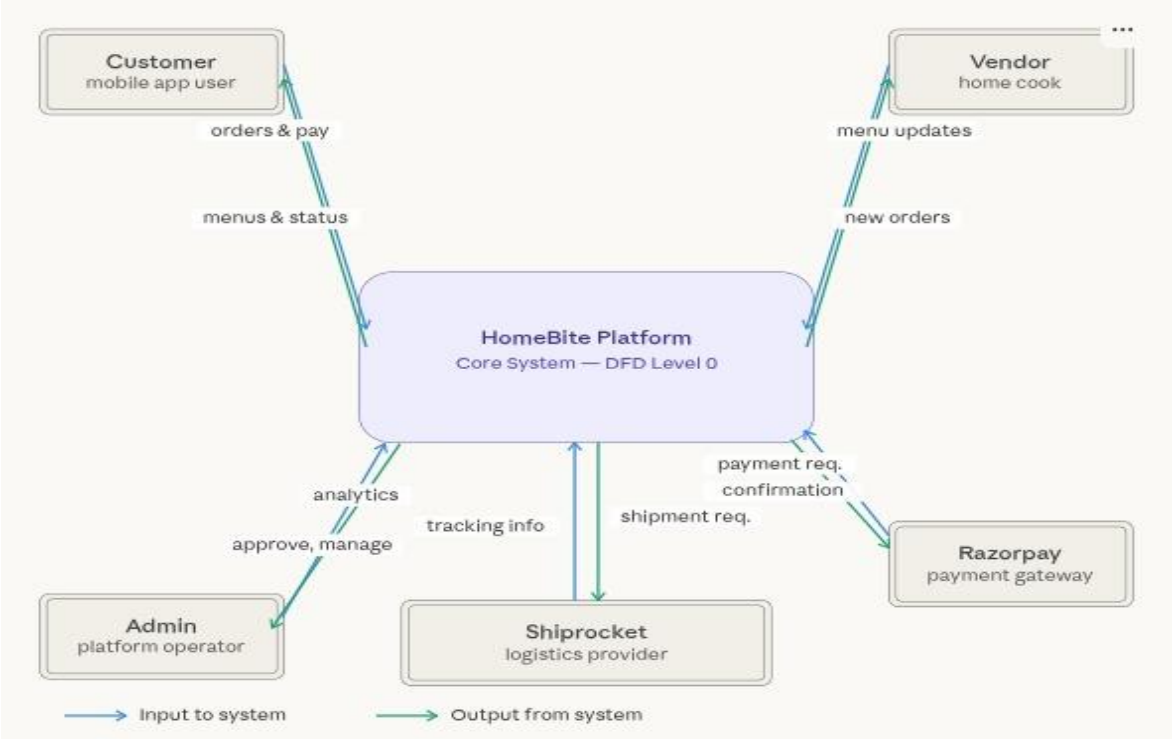
4.2 Architecture Diagram



4.3 Module Interaction Diagram

Every HTTP request processed by the HomeBite backend traverses a consistent, layered middleware pipeline before reaching the business logic layer. The module interaction flow is as follows:

7. The HTTP request arrives at the Express.js router.
8. The authentication middleware verifies the JWT access token, extracts the user payload, and attaches it to the request object.
9. The role guard middleware verifies that the authenticated user possesses the role required to access the requested endpoint.
10. The Joi validation middleware validates the request body against the declared schema, returning HTTP 400 with a structured errors array on validation failure.
11. The controller receives the validated request, invokes the corresponding service function, and formats the response.
12. The service function executes business logic, interacts with the Prisma client to perform database operations, and returns a result.
13. The Prisma client executes parameterised SQL queries against the MySQL database and returns typed result objects.



4.4 UML Diagrams

4.4.1 Use Case Diagram

The use case diagram illustrates the complete set of interactions available to each of the three actor types — Customer, Vendor, and Administrator — within the HomeBite system. Customer actors can perform: user registration and login, vendor and food browsing, cart management, order placement, payment processing, delivery tracking, and review submission. Vendor actors can perform: vendor profile management, food item CRUD operations, order acceptance and rejection, order status updates, and delivery initiation. Administrator actors can perform:

vendor approval and suspension, user management, platform-wide order visibility, and analytics access

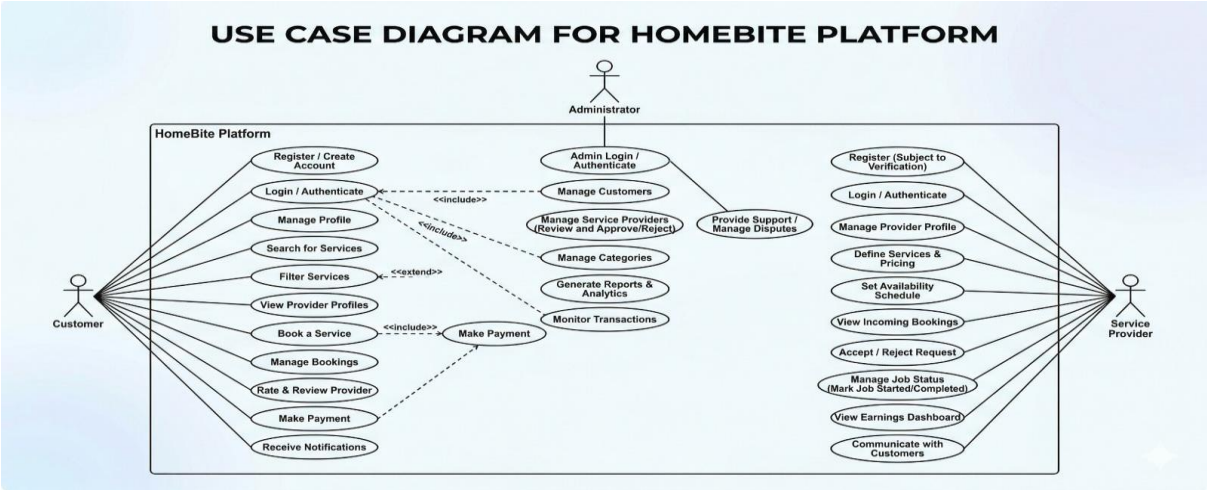
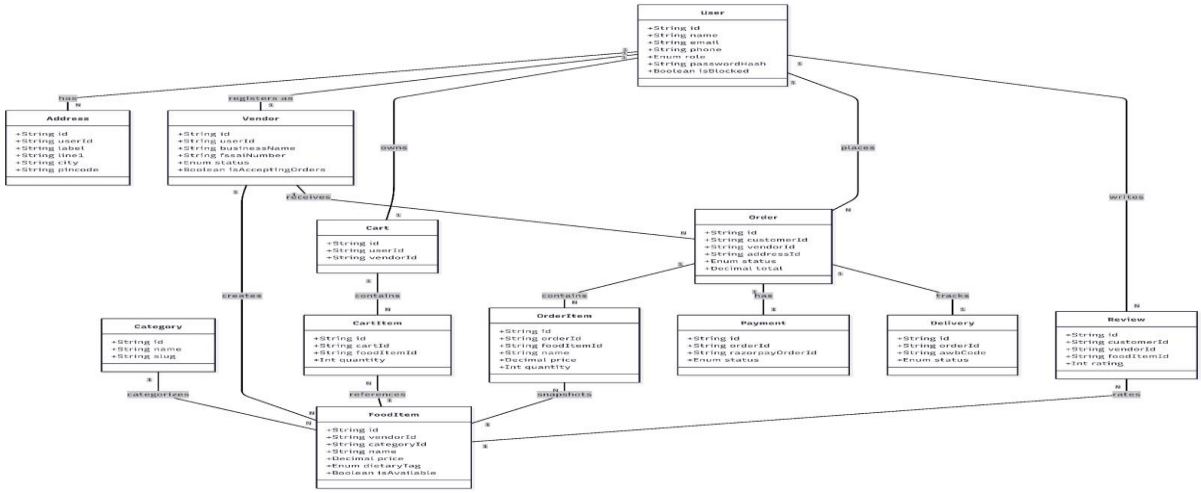


Figure 4.3: Use Case Diagram of HomeBite

4.4.2 Class Diagram

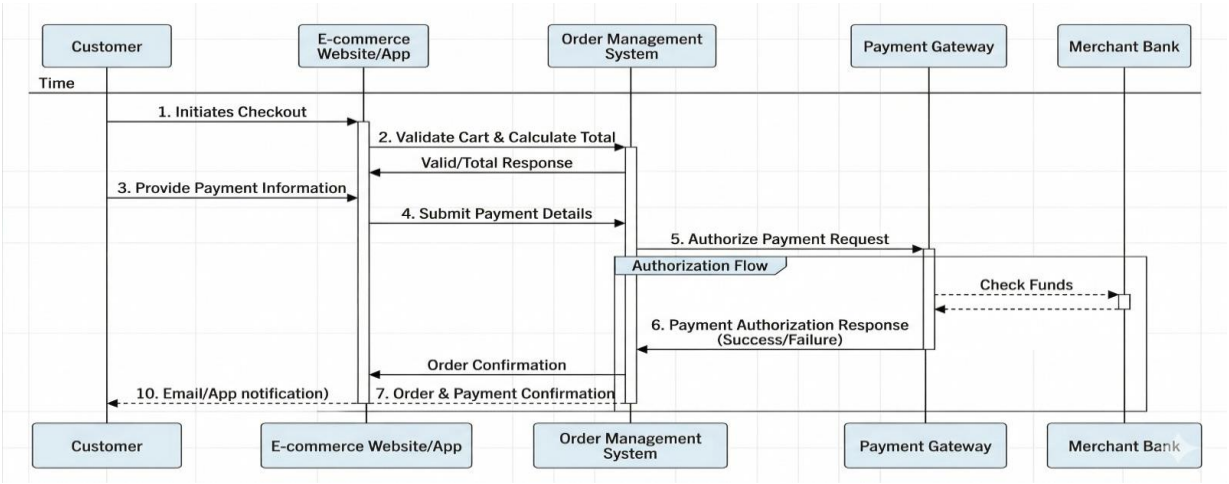
The class diagram represents the primary domain entities in the HomeBite system and their relational associations. The key entities are: User, Vendor, FoodItem, Category, Cart, CartItem, Order, OrderItem, Payment, Delivery, Review, Address, and OtpToken. Key relationships include: a one-to-one relationship between User and Vendor (one vendor profile per user), a one-to-many relationship between Vendor and FoodItem, a many-to-one relationship between FoodItem and Category, and one-to-one relationships between Order and Payment, and Order and Delivery.



4.4.3 Sequence Diagram – Order Placement and Payment

The sequence diagram illustrates the chronological flow of interactions between the Customer, the mobile application, the HomeBite backend API, the MySQL database, and the Razorpay payment gateway during the critical order placement and payment verification workflow. The sequence begins with the customer initiating an order, proceeds through cart validation and

order creation with price snapshotting, continues through Razorpay payment order creation and client-side payment completion, and concludes with the server-side HMAC-SHA256 payment signature verification that transitions the order to CONFIRMED status.



4.5 Data Flow Diagrams

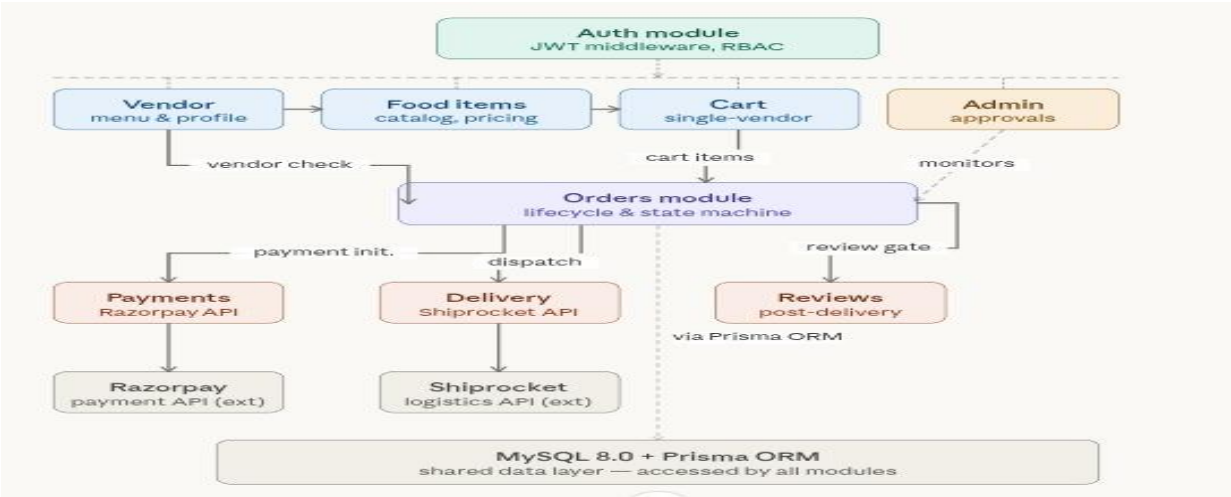
4.5.1 Context Level DFD

The context level (Level 0) Data Flow Diagram depicts HomeBite as a single process interacting with five external entities: Customer, Vendor, Administrator, Razorpay, and Shiprocket. Customers provide registration data, food orders, and payment information, and receive food listings, order confirmations, and delivery tracking. Vendors provide business registration data and order status updates, and receive order notifications and analytics. Administrators provide approval decisions and management directives, and receive platform analytics and reports. Razorpay receives payment order creation requests and returns payment

order identifiers and payment verification data. Shiprocket receives shipment creation *Figure 4.6: Context Level Data Flow Diagram*

4.5.2 Level 1 DFD

The Level 1 DFD decomposes the single HomeBite process into eight primary sub-processes:



User Authentication, Vendor Management, Food Catalogue Management, Cart and Order Processing, Payment Processing, Delivery Tracking, Review Management, and Admin Analytics. Each sub-process is connected to the appropriate data stores (User Database, Vendor Database, Food Item Repository, Order Storage, Payment Records, Delivery Records, Review Storage, and Analytics Data Store) and to the relevant external entities.

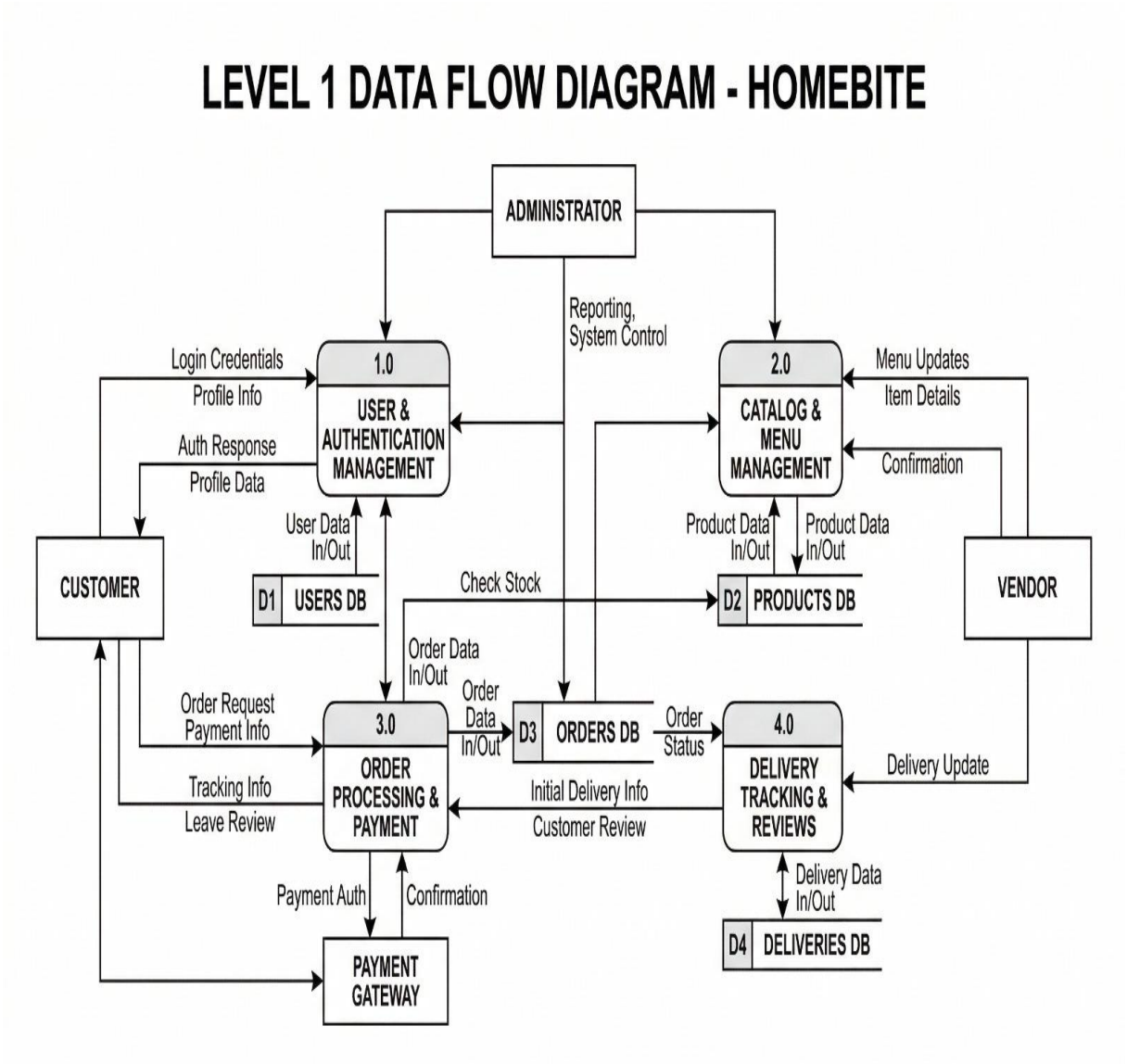


Figure 4.7: Level 1 Data Flow Diagram

Chapter 5

DATABASE DESIGN

5.1 Database Overview

The HomeBite database is implemented in MySQL 8.0, chosen for its strong relational integrity enforcement, mature ecosystem, widespread industry adoption, and excellent support for the utf8mb4 character set which enables full Unicode support including emoji in food item descriptions. All database access is mediated exclusively through Prisma ORM, which serves as the single source of truth for schema definitions through the prisma/schema.prisma file. All monetary values are stored as DECIMAL(10, 2) to avoid the floating-point precision errors that arise with IEEE 754 floating-point types. All primary keys use CUID (Collision-resistant Unique Identifier) format generated by Prisma's cuid() function, which provides both global uniqueness and lexicographic sortability.

5.2 Table Definitions

5.2.1 users

Column	Type	Constraints	Description
id	VARCHAR (CUID)	PRIMARY KEY	Unique user identifier
name	VARCHAR	NOT NULL	Full name of the user
email	VARCHAR	UNIQUE, NOT NULL	Login identifier and contact
phone	VARCHAR	UNIQUE, NULLABLE	Optional phone number
passwordHash	TEXT	NOT NULL	bcrypt-hashed password
role	ENUM	NOT NULL, DEFAULT CUSTOMER	CUSTOMER / VENDOR / ADMIN
isBlocked	BOOLEAN	DEFAULT false	Administrator block flag
refreshToken	TEXT	NULLABLE	Current active refresh token (single session)
createdAt	DATETIME	DEFAULT now()	Account creation timestamp

5.2.2 vendors

Column	Type	Constraints	Description
id	VARCHAR (CUID)	PRIMARY KEY	Unique vendor identifier
userId	VARCHAR	FK → users.id, UNIQUE	One vendor profile per user
businessName	VARCHAR	NOT NULL	Kitchen or brand name
fssaiNumber	VARCHAR	UNIQUE, NULLABLE	Food safety licence number
status	ENUM	DEFAULT PENDING	PENDING / APPROVED / REJECTED / SUSPENDED
isAcceptingOrders	BOOLEAN	DEFAULT false	Toggle: open/closed for orders
lat / lng	FLOAT	NULLABLE	Geolocation coordinates

5.2.3 food_items

Column	Type	Constraints	Description
id	VARCHAR (CUID)	PRIMARY KEY	Unique food item identifier
vendorId	VARCHAR	FK → vendors.id CASCADE	Owning vendor
categoryId	VARCHAR	FK → categories.id	Food category association
name	VARCHAR	NOT NULL	Food item name
price	DECIMAL(10,2)	NOT NULL	Base selling price
discountedPrice	DECIMAL(10,2)	NULLABLE	Sale/promotional price
dietaryTag	ENUM	DEFAULT VEG	VEG / NON_VEG / VEGAN / JAIN / GLUTEN_FREE
isAvailable	BOOLEAN	DEFAULT true	Toggle item availability
isDeleted	BOOLEAN	DEFAULT false	Soft delete flag

5.2.4 orders

Column	Type	Constraints	Description
id	VARCHAR (CUID)	PRIMARY KEY	Unique order identifier

customerId	VARCHAR	FK → users.id	Placing customer
vendorId	VARCHAR	FK → vendors.id	Receiving vendor
addressId	VARCHAR	FK → addresses.id	Delivery address snapshot
status	ENUM	DEFAULT PENDING_PAYMENT	Order lifecycle state
subtotal	DECIMAL(10,2)	NOT NULL	Sum of order item totals
total	DECIMAL(10,2)	NOT NULL	Final charged amount including delivery

5.2.5 order_items (with Price Snapshotting)

Column	Type	Constraints	Description
id	VARCHAR (CUID)	PRIMARY KEY	Unique order item identifier
orderId	VARCHAR	FK → orders.id CASCADE	Parent order
foodItemId	VARCHAR	FK → food_items.id	Reference to menu item
name	VARCHAR	NOT NULL	SNAPSHOT: item name at order time
price	DECIMAL(10,2)	NOT NULL	SNAPSHOT: price at order time
quantity	INT	NOT NULL	Quantity ordered
total	DECIMAL(10,2)	NOT NULL	price × quantity

5.2.6 payments

Column	Type	Constraints	Description
id	VARCHAR (CUID)	PRIMARY KEY	Unique payment identifier
orderId	VARCHAR	FK, UNIQUE	One payment record per order
razorpayOrderId	VARCHAR	UNIQUE, NULLABLE	Razorpay order identifier
razorpayPaymentId	VARCHAR	UNIQUE, NULLABLE	Razorpay payment identifier
razorpaySignature	TEXT	NULLABLE	HMAC-SHA256 signature for verification
amount	DECIMAL(10,2)	NOT NULL	Charged amount in INR

status	ENUM	DEFAULT PENDING	PENDING / SUCCESS / FAILED

5.2.7 deliveries

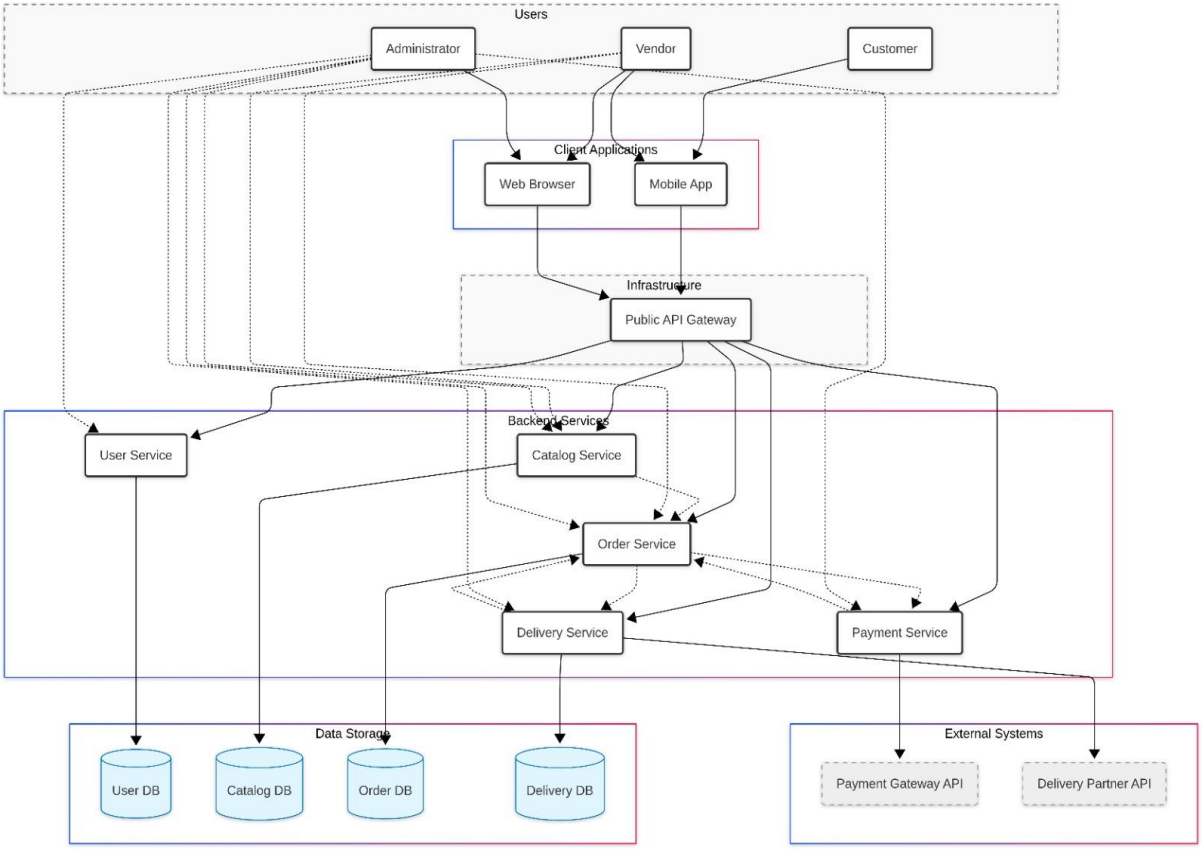
Column	Type	Constraints	Description
id	VARCHAR (CUID)	PRIMARY KEY	Unique delivery identifier
orderId	VARCHAR	FK, UNIQUE	One delivery record per order
awbCode	VARCHAR	NULLABLE	Air Waybill number from Shiprocket
courierName	VARCHAR	NULLABLE	Assigned courier partner name
trackingUrl	TEXT	NULLABLE	Live delivery tracking URL
status	ENUM	DEFAULT NOT_INITIATED	Delivery lifecycle state

5.3 Entity-Relationship Diagram

The Entity-Relationship Diagram illustrates the structural relationships between all primary database entities in the HomeBite system. The key relationships are:

- USERS to ADDRESSES: One-to-many. Each user may store multiple delivery addresses.
- USERS to VENDORS: One-to-one. Each user account may have at most one vendor profile.
- VENDORS to FOOD_ITEMS: One-to-many. Each approved vendor may list multiple food items.
- FOOD_ITEMS to CATEGORIES: Many-to-one. Each food item belongs to exactly one category.
- USERS to CART to CART_ITEMS: One-to-one to one-to-many. Each customer has one active cart containing multiple cart items referencing food items.
- USERS to ORDERS to ORDER_ITEMS: One-to-many to one-to-many. Each customer may place multiple orders, each containing multiple order items with snapshotted prices.
- ORDERS to PAYMENTS: One-to-one. Each order has exactly one associated payment record.

- ORDERS to DELIVERIES: One-to-one. Each order has exactly one associated delivery tracking record.
- USERS to REVIEWS to FOOD_ITEMS: Many-to-many bridge. Reviews are submitted by customers for specific food items, restricted to delivered orders.



5.4 Key Design Decisions

Decision 1: Price Snapshotting in order_items

The name and price columns in the order_items table are denormalised copies of the food item's name and price, captured at the moment of order placement. This design decision ensures that any subsequent changes made by a vendor to a food item's price — whether an increase or a decrease — do not retroactively affect historical order records. This is critical for financial accuracy, customer trust, and dispute resolution.

Decision 2: Single-Vendor Cart Enforcement

The Cart model stores a vendorId field that is set when the first item is added to the cart. Any subsequent attempt to add an item from a different vendor is rejected at the service layer with an HTTP 400 error and a descriptive message, preventing the creation of cross-vendor orders that would be operationally impossible to fulfil.

Decision 3: Soft Deletes for food_items and reviews

Both the food_items and reviews tables implement soft deletion through an isDeleted boolean flag. When a food item is soft-deleted, existing order_items records that reference it retain their foreign key integrity while the item is no longer returned in public listings. This design preserves the historical accuracy of past orders and avoids orphaned foreign key references.

Decision 4: Single Active Session per User

The refreshToken field is stored directly in the users table as a single column, enforcing a single active session per user account. When a new login occurs or when the refresh token is rotated, the new token overwrites the previous value in the users table, automatically invalidating the previous session. This design simplifies session management while providing an effective mechanism for single-session enforcement.

Chapter 6

TECHNOLOGY STACK AND IMPLEMENTATION

6.1 Backend Technologies

6.1.1 Node.js

Node.js 18 LTS provides the JavaScript runtime environment for the HomeBite backend. Its non-blocking, event-driven I/O model is particularly well-suited to a marketplace API that must handle many concurrent short-duration requests — such as product listing queries, cart operations, and order status checks — without the thread-contention overhead of traditional thread-per-request server architectures. The extensive npm ecosystem provides battle-tested, production-ready packages for every requirement in this project, eliminating the need to implement foundational utilities from scratch.

6.1.2 Express.js

Express.js is the de-facto standard minimal web application framework for Node.js. Its middleware model maps cleanly onto the layered architecture employed in HomeBite, where each route applies a sequential chain of authentication, role-based access control, and validation middleware before reaching the controller. Express.js's routing system, combined with its modular Router objects, enables the clean feature-based module structure used in HomeBite. The framework's minimalism ensures that no unnecessary abstractions are imposed on the application's structure.

6.1.3 Prisma ORM

Prisma was selected as the ORM for three primary reasons. First, its schema-first approach means that the prisma/schema.prisma file is the single source of truth for all database table definitions, column types, enumerations, and relational constraints. Migrations are auto-generated and version-controlled through the prisma migrate workflow. Second, the generated Prisma client provides full TypeScript type safety and IDE autocomplete for all database queries, eliminating an entire class of runtime errors caused by incorrect field names or types. Third, Prisma Studio — a built-in visual database browser — significantly accelerates development iteration and debugging.

6.1.4 MySQL 8.0

MySQL 8.0 was selected as the relational database management system for HomeBite based on its reliability, strong enforcement of relational integrity through foreign key constraints and transactions, widespread industry adoption with mature operational tooling, and native support for the utf8mb4 character set enabling full Unicode including emoji in food item names and descriptions. The relational model is well-suited to HomeBite's domain, where entities such as orders, order items, users, vendors, and payments have well-defined, stable relationships that benefit from referential integrity enforcement.

6.1.5 Backend Library Summary

Table 6.1 summarises the key supporting libraries used in the HomeBite backend.

Library	Version	Purpose
jsonwebtoken	9.0.2	JWT signing, verification, and payload extraction
bcryptjs	2.4.3	Secure password hashing with configurable salt rounds
joi	17.13.1	Declarative request body and parameter validation
helmet	7.1.0	Security-relevant HTTP response headers
cors	2.8.5	Cross-origin resource sharing policy enforcement
express-rate-limit	7.3.1	IP-based rate limiting (200 requests / 15 minutes)
multer	1.4.5-lts.1	Multipart form-data file upload handling
razorpay	2.9.2	Razorpay Node.js SDK for payment order creation
compression	1.7.4	Gzip response compression for bandwidth efficiency
morgan	1.10.0	HTTP request logging middleware
dotenv	16.x	Environment variable loading from .env files

6.2 Mobile Application Technologies

6.2.1 React Native and Expo SDK 51

React Native enables the development of a single JavaScript/TypeScript codebase that produces fully native mobile applications for both Android and iOS platforms. This cross-

platform approach eliminates the cost and complexity of maintaining two separate native codebases while delivering a native user experience on both platforms. Expo SDK 51 significantly reduces the initial setup overhead and provides managed access to native device capabilities — including secure storage, image picking, and push notifications — through well-maintained JavaScript packages that abstract platform-specific implementations.

6.2.2 Expo Router v3.5

Expo Router v3.5 brings file-system-based routing — conceptually similar to Next.js for web — to React Native applications. Routes are defined by the file system structure within the app/ directory, providing automatic deep linking support, type-safe navigation, and a clear separation between the three role-based navigation stacks: (customer), (vendor), and (admin). Route groups enable these stacks to coexist in a single application while maintaining independent layout and authentication requirements for each role.

6.2.3 Zustand for State Management

Zustand was selected over Redux Toolkit for global state management based on its minimal boilerplate, excellent TypeScript support, and simple mental model. Two Zustand stores are maintained in HomeBite: the authStore manages the user session, access token, authenticated user data, and authentication state transitions (login, logout, token refresh); the cartStore manages cart items, computed totals, and all cart mutation actions (add, update, remove, clear). Zustand's shallow equality subscription mechanism prevents unnecessary re-renders when unrelated state slices change.

6.2.4 Mobile Library Summary

Table 6.2 summarises the key supporting libraries used in the HomeBite mobile application.

Library	Version	Purpose
axios	1.7.2	HTTP client with request/response interceptors for token injection
react-hook-form	7.52.1	Performant, uncontrolled form state management
zod	3.23.8	TypeScript-first runtime schema validation
zustand	4.x	Lightweight global state management for auth and cart
react-native-reanimated	3.10.1	Smooth, native-thread UI animations
expo-secure-store	13.x	Encrypted native storage for

		refresh tokens
expo-image-picker	15.x	Camera roll and camera access for food item images
react-native-toast-message	2.2.0	Non-intrusive user feedback toast notifications

6.3 Project Structure

The HomeBite project is organised into two independently deployable components: homebite-backend and homebite-mobile. The backend follows a feature-based module structure in which each domain of business logic (auth, vendor, food-items, cart, orders, payments, delivery, reviews, admin) is self-contained within its own directory, encapsulating routes, controller, and service files. Shared concerns — including authentication middleware, role guard middleware, Joi validation middleware, and the asyncHandler utility — reside in a dedicated middleware/ directory accessible to all modules. The Prisma schema and migrations reside in the prisma/ directory.

The mobile application is structured with Expo Router's file-based routing under the app/ directory, subdivided into role-based route groups: (customer)/, (vendor)/, and (admin)/. Zustand stores reside in a stores/ directory, API service functions in a services/ directory, and reusable UI components in a components/ directory.

6.4 API Design

6.4.1 Design Principles

All HomeBite API endpoints adhere to the following consistent design principles:

- All endpoints are versioned and prefixed with /api/v1 to support non-breaking future API evolution.
- All responses follow a standardised JSON envelope structure: { success: boolean, message: string, data: object | array | null }.
- Validation errors return HTTP 400 with a structured errors array detailing all field-level validation failures.
- Authentication failures return HTTP 401; authorisation failures return HTTP 403; resource not found returns HTTP 404.
- All list endpoints support pagination through page and limit query parameters.

- Successful creation operations return HTTP 201; successful updates return HTTP 200; successful deletions return HTTP 200 with a confirmation message.

6.4.2 Authentication Endpoints

The authentication module provides endpoints for user registration, login, token refresh, profile retrieval, logout, and password management for all three user roles. Each role (Admin, Teacher, Student) has its own base path to support role-specific registration workflows and data requirements.

6.4.3 Vendor Endpoints

The vendor module provides endpoints for vendor application submission (POST /api/v1/vendors/register), vendor profile retrieval and update (GET/PUT /api/v1/vendors/profile for the authenticated vendor), public vendor listing (GET /api/v1/vendors), and individual vendor menu retrieval (GET /api/v1/vendors/:id). The vendor registration endpoint accepts a multipart form-data request containing the business profile data and a logo image file processed by Multer.

6.4.4 Order Endpoints

The order module implements the complete order lifecycle through dedicated endpoints. Order placement (POST /api/v1/orders) validates the cart, snapshots prices, creates the order and associated delivery record, and clears the cart atomically within a Prisma transaction. Order status transitions are managed through role-specific endpoints: vendors accept (POST .../accept) or reject (POST .../reject) CONFIRMED orders, update status through preparation stages (PATCH .../status), and customers cancel (POST .../cancel) orders at eligible status stages.

6.4.5 Payment Endpoints

The payment module provides three endpoints: Razorpay order creation (POST /api/v1/payments/razorpay/order), which creates a Razorpay order object and returns the razorpayOrderId to the client; payment verification (POST /api/v1/payments/razorpay/verify), which performs server-side HMAC-SHA256 signature verification and transitions the order to CONFIRMED on success or PAYMENT_FAILED on signature mismatch; and payment record retrieval (GET /api/v1/payments/:orderId).

6.4.6 Admin Endpoints

The admin module provides comprehensive platform management through endpoints for: vendor listing with status filtering (GET /api/v1/admin/vendors), vendor status update for approval, rejection, and suspension (PATCH /api/v1/admin/vendors/:id/status), platform-wide order listing (GET /api/v1/admin/orders), user management (GET /api/v1/admin/users), and platform analytics aggregation (GET /api/v1/admin/analytics) returning aggregate statistics including total users, vendors, orders, revenue, and recent activity.

Chapter 7

SECURITY DESIGN

7.1 Authentication Architecture

HomeBite implements a stateless JWT dual-token authentication model designed to balance security (through short-lived tokens) and usability (through transparent silent refresh). This model eliminates the need for server-side session storage, enabling horizontal scaling of the backend without session affinity requirements.

7.1.1 Access Token

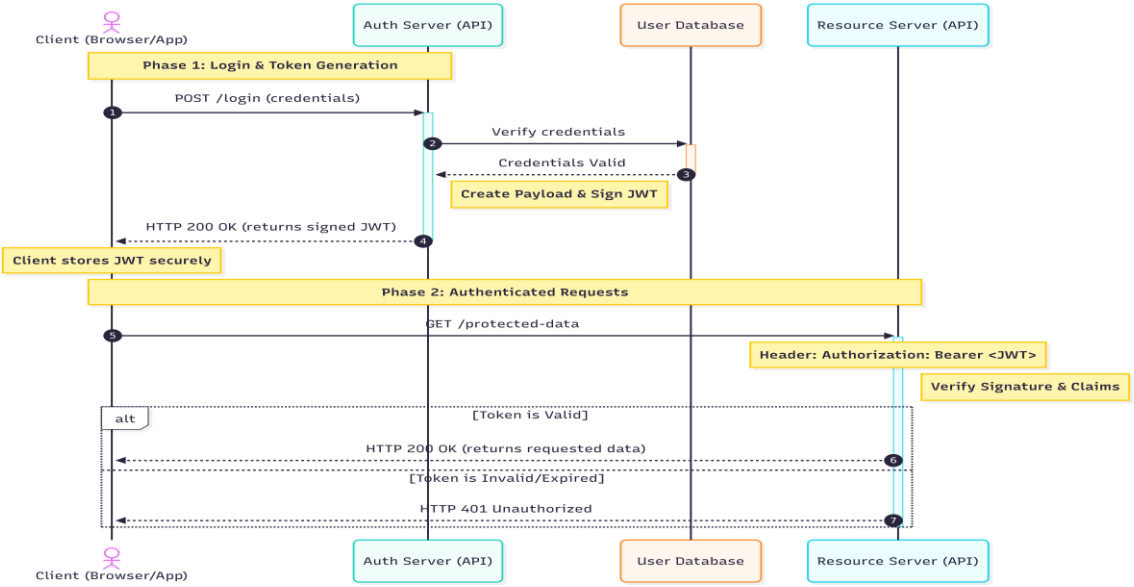
Access tokens are signed using the HS256 (HMAC-SHA256) algorithm. The token payload contains: { userId, role, iat (issued-at), exp (expiry) }. Tokens have a short 15-minute expiry window, limiting the window of exposure in the event of token interception. On the client side, access tokens are stored exclusively in Zustand memory and are never persisted to device storage or localStorage, preventing token theft through storage access attacks. Every authenticated API request includes the access token in the Authorization: Bearer header.

7.1.2 Refresh Token

Refresh tokens are signed using the HS256 algorithm with a 7-day expiry, providing a convenient session duration without requiring the user to re-authenticate frequently. On the client side, refresh tokens are persisted to device secure storage via expo-secure-store, which leverages the native Android Keystore System and iOS Keychain, providing hardware-backed encryption for stored tokens. On the server side, the current valid refresh token is stored in the refreshToken column of the users table, enforcing a single active session per user account.

Refresh token rotation is implemented as follows: when the client submits a valid refresh token to obtain a new token pair, the backend verifies the submitted token against the value stored in the database, issues a new access token and refresh token pair, and immediately overwrites the stored refresh token with the new value, invalidating the previous token. If the client subsequently submits the old (now invalidated) refresh token — which would indicate a potential token theft scenario where an attacker obtained an old token — the server detects the mismatch with the stored token, rejects the request with HTTP 401, and can optionally revoke the current token to force a complete re-authentication.

Figure 7.1: JWT Dual-Token Authentication Flow Diagram



7.2 Role-Based Access Control (RBAC)

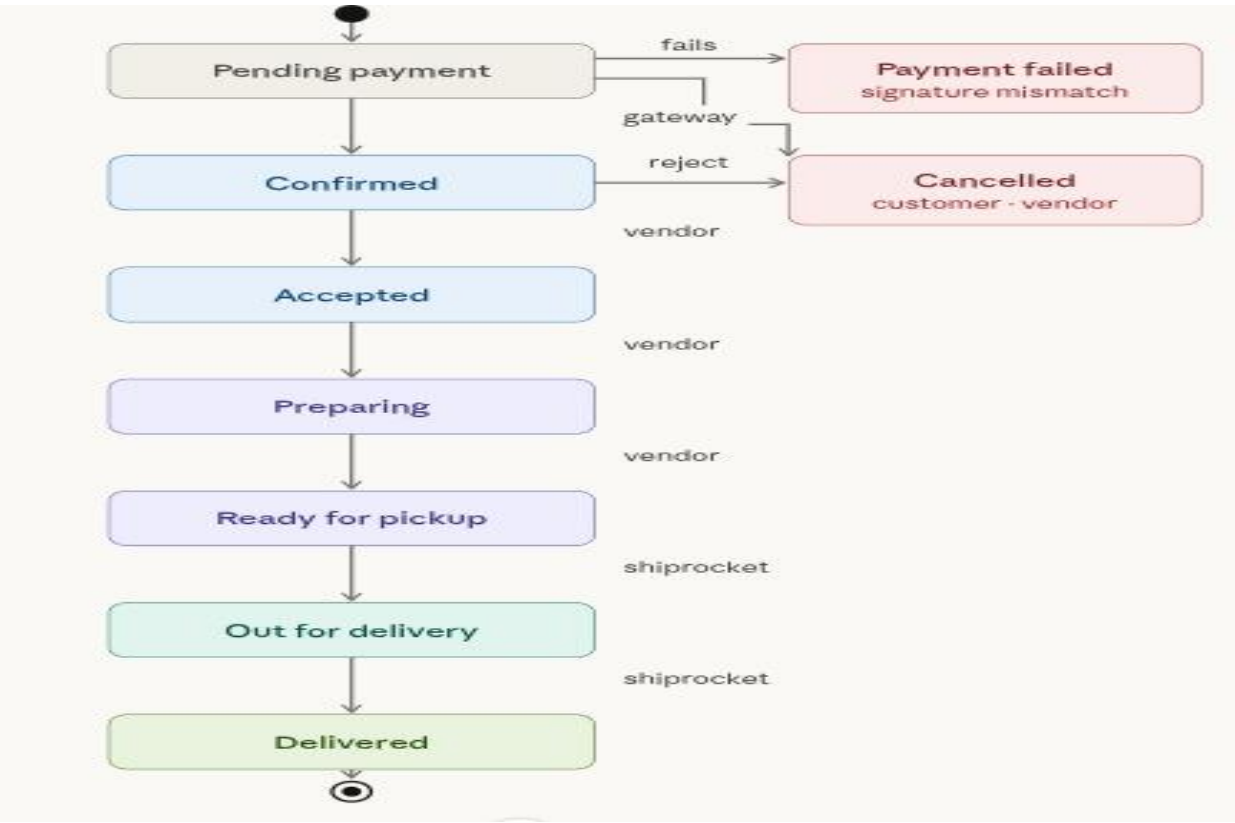
HomeBite implements a role-based access control system through a dedicated roleGuard middleware that is applied after the authentication middleware in the route pipeline. The authentication middleware verifies the access token and attaches the decoded user payload — including the role field — to the request object. The roleGuard middleware then checks whether the user's role is included in the array of roles permitted to access the requested endpoint, returning HTTP 403 Forbidden if the role requirement is not met.

Table 7.1 summarises the access levels and permitted modules for each role in the HomeBite RBAC system.

Role	Access Level	Accessible Modules and Operations
CUSTOMER	Base	Registration, login, vendor/food browsing, cart management, order placement, payment processing, delivery tracking (read), review submission (post-delivery only)
VENDOR	Elevated	All CUSTOMER capabilities plus: food item CRUD (own items), order management (received orders), delivery initiation, vendor profile management
ADMIN	Superuser	All platform capabilities: vendor

		approval/rejection/suspension, user management (create, block, delete), all orders visibility, platform analytics

Figure 7.2: Order State Machine – Status Transition Diagram



7.3 Payment Security

Payment security is enforced through server-side HMAC-SHA256 signature verification of Razorpay payment callbacks. When a customer completes a payment through the Razorpay checkout UI, Razorpay provides the client application with three values: razorpayOrderId,

razorpayPaymentId, and razorpaySignature. The client submits all three values to the HomeBite backend's payment verification endpoint.

The backend independently computes the expected signature as the HMAC-SHA256 hash of the string (razorpayOrderId | '|' | razorpayPaymentId) using the RAZORPAY_KEY_SECRET as the key. If the computed expected signature matches the submitted razorpaySignature, the payment is confirmed as authentic and the order status is transitioned to CONFIRMED. If the signatures do not match — indicating a potential payment fabrication attempt — the payment status is set to FAILED and the order transitions to PAYMENT_FAILED. This server-side verification ensures that no client-side manipulation of payment data can result in a fraudulent order confirmation.

7.4 Additional Security Measures

Table 7.2 summarises the additional security measures implemented across the HomeBite system.

Measure	Implementation
HTTP Security Headers	Helmet middleware sets X-Frame-Options, HSTS, Content-Security-Policy, X-XSS-Protection, and Referrer-Policy headers
Rate Limiting	express-rate-limit enforces 200 requests per IP per 15-minute window; HTTP 429 returned on violation
CORS	Explicit ALLOWED_ORIGINS whitelist in cors middleware; unlisted origins receive CORS error response
Password Hashing	bcryptjs with 10 salt rounds; passwordHash never returned in any API response
Input Validation	Joi declarative schemas with stripUnknown: true on all request bodies and parameters
Sensitive Data Exclusion	passwordHash and refreshToken excluded from all API responses via Prisma select clauses
Environment Variables	All secrets (JWT_SECRET, DB_URL, RAZORPAY_KEY_SECRET) stored in .env; .gitignore excludes .env from version control
SQL Injection Prevention	Prisma ORM uses parameterised queries exclusively; no raw SQL string interpolation

The combination of these security measures provides a comprehensive, defence-in-depth security posture for the HomeBite platform, addressing the most common web application

security risks identified in the OWASP Top 10, including injection attacks, broken authentication, sensitive data exposure, broken access control, and security misconfiguration.

Chapter 8

DEPLOYMENT ARCHITECTURE

8.1 Production Environment Components

The HomeBite production environment is designed for reliability, maintainability, and scalability. Table 8.1 summarises the components of the production environment and their responsibilities.

Component	Technology	Responsibility
Mobile Application	Expo EAS Build + App Stores	Android APK / iOS IPA distribution
Backend API Server	Node.js on cloud VM / container	REST API serving and business logic
Database	MySQL (managed cloud service)	Relational data persistence
File Storage	AWS S3	Food item images and media files
CDN	CloudFront / Cloudflare	Image delivery acceleration and SSL termination
Process Manager	PM2	Process restart, clustering, and structured logging
Container Runtime	Docker / Docker Compose	Application containerisation and orchestration

8.2 Docker Configuration

HomeBite is containerised using Docker to ensure consistent, reproducible deployments across development, staging, and production environments. The Dockerfile for the backend service uses a lightweight Node.js 18 Alpine base image, installs only production dependencies using npm ci --only=production to minimise the final image size, copies the application source code, generates the Prisma client for the target platform, exposes port 5000, and sets the container start command.

Docker Compose orchestrates the multi-container setup for local development and production deployment. The composition includes two services: the db service running the official MySQL 8.0 image with a persistent named volume for data storage, and the backend service built from the Dockerfile, configured through an environment file, dependent on the db service, and

configured for automatic restart on failure. The depends_on directive ensures the database service is healthy before the backend service attempts to connect.

8.3 CI/CD Deployment Pipeline

HomeBite employs a Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions to automate the build, test, and deployment workflow. The pipeline is triggered automatically on every push to the main branch and consists of the following stages:

14. Code Quality and Linting: ESLint and Prettier are run against the codebase to enforce coding standards and identify potential issues before deployment.
15. Automated Testing: The Jest test suite is executed to validate all unit tests and integration tests. A failed test suite blocks deployment and notifies the development team.
16. Docker Image Build: A production Docker image is built using the Dockerfile and tagged with the Git commit SHA for traceability.
17. Image Push to Registry: The built image is pushed to a container registry (AWS ECR or Docker Hub), making it available for deployment to the production environment.

Figure 8.1: CI/CD Deployment Pipeline

8.4 Mobile App Distribution

The HomeBite mobile application is built and distributed using Expo Application Services (EAS). EAS Build produces fully native Android APK / AAB and iOS IPA files in the cloud without requiring the development machine to have Xcode or Android Studio configured. The EAS workflow is as follows:

18. Configure EAS Build profiles for development, staging, and production environments using the eas.json configuration file.
19. Trigger cloud builds for each platform: eas build --platform android --profile production and eas build --platform ios --profile production.
20. Upon successful build completion, submit the built binary to the respective app stores using eas submit --platform android and eas submit --platform ios.
21. Over-the-Air (OTA) updates for JavaScript bundle changes — which do not affect native code — can be deployed instantaneously through the Expo Updates service, without requiring a full app store submission and review cycle.

This deployment approach ensures that HomeBite users on both Android and iOS platforms always have access to the latest version of the application with minimal deployment friction for the development team.

Chapter 9

RESULTS AND TESTING

9.1 Testing Methodology

The HomeBite system underwent a comprehensive, multi-layered testing programme covering unit testing, integration testing, system testing, performance testing, and security testing. The objective of this testing programme was to validate all functional requirements, verify the correct enforcement of the order state machine, confirm security posture against common web application vulnerabilities, and establish performance baselines under various concurrent load conditions.

9.1.1 Unit Testing

Individual service functions were tested in isolation using Jest as the test runner and Supertest as the HTTP assertion library for endpoint testing. Mock implementations of the Prisma client were used to eliminate database dependencies in unit tests, enabling fast execution and deterministic outcomes. Key areas covered by unit tests include: JWT token generation and verification logic, bcrypt password hashing and comparison, Joi validation schema correctness, HMAC-SHA256 payment signature computation, and order status transition validation.

9.1.2 Integration Testing

Integration tests were conducted against a dedicated test database instance to verify the correct interaction between system components. Integration testing covered: frontend-to-backend API communication via Axios interceptors, database CRUD operations through the Prisma client, authentication and authorisation flows end-to-end, single-vendor cart enforcement across multiple API calls, order placement with price snapshotting and cart clearance, and payment signature verification integrated with order status transitions.

9.1.3 System Testing

System testing exercised complete end-to-end user journeys covering: the full customer journey from registration through vendor browsing, cart building, order placement, payment, delivery tracking, and review submission; the full vendor journey from onboarding application through admin approval, menu management, order acceptance, status updates, and delivery

initiation; and the complete administrator workflow covering vendor approval, user management, and analytics access.

9.2 Test Cases

Table 9.1 presents the sample test cases executed during the testing programme, along with expected results, actual results, and pass/fail status.

TC-ID	Test Description	Expected Result	Actual Result	Status
TC-01	User registration with valid data	Account created; 201 response	Account created	PASS
TC-02	Login with correct credentials	JWT token pair issued	Tokens issued	PASS
TC-03	Login with incorrect password	HTTP 401 Unauthorized	401 returned	PASS
TC-04	Add item from second vendor to cart	HTTP 400 error	400 returned	PASS
TC-05	Place order with valid cart	Order created; cart cleared; price snapshot saved	All correct	PASS
TC-06	Payment with invalid HMAC signature	Order → PAYMENT_FAILED	Status updated	PASS
TC-07	Payment with valid HMAC signature	Order → CONFIRMED	Status updated	PASS
TC-08	Vendor accepts order	Order status → ACCEPTED	Status updated	PASS
TC-09	Vendor rejects order without reason	HTTP 400 validation error	400 returned	PASS
TC-10	Customer reviews undelivered order	HTTP 403 Forbidden	403 returned	PASS
TC-11	Refresh token rotation	New token pair issued; old invalidated	Both correct	PASS
TC-12	Reuse of invalidated refresh token	HTTP 401 rejected	401 returned	PASS
TC-13	Admin approves vendor	Vendor status → APPROVED	Status updated	PASS
TC-14	Blocked user attempts login	HTTP 403 Forbidden	403 returned	PASS

TC-15	Access protected endpoint without token	HTTP 401 Unauthorized	401 returned	PASS

9.3 Performance Testing

Performance testing was conducted using Apache JMeter to simulate concurrent user load against the HomeBite API. The test targeted the most computationally intensive endpoints — vendor listing, order placement, and cart management — across four load scenarios representing normal operating conditions, medium load, high load, and peak load. Table 9.2 presents the results.

Test Scenario	Concurrent Users	Avg Response (ms)	CPU Usage (%)
Normal Load	50	180	28
Medium Load	100	320	48
High Load	200	620	71
Peak Load	300	1100	83

The results indicate that HomeBite performs acceptably under normal load (up to 100 concurrent users) with response times well within the acceptable threshold of 500 ms. Under high and peak load conditions, response times increase as expected due to MySQL connection pool saturation and CPU contention. These findings indicate that horizontal scaling of the backend — enabled by its stateless architecture — would be required to sustain performance at peak load in a production environment serving a large user base.

9.4 Security Testing

Security testing was conducted to verify that all implemented security measures function correctly under adversarial conditions. The following tests were performed:

- Authentication Bypass: Attempts to access protected endpoints without a valid JWT access token consistently returned HTTP 401 Unauthorized, confirming correct enforcement of the authentication middleware.
- SQL Injection: Attempts to inject SQL through request body fields and query parameters were neutralised by Prisma's parameterised query engine, which never interpolates raw user input into SQL strings.
- XSS Attempts: Content-Security-Policy headers set by Helmet middleware prevented script injection payloads from being executed in browser contexts.

- CSRF: The stateless JWT authentication model (using Authorization headers rather than cookies) provides inherent immunity to Cross-Site Request Forgery attacks.
- Rate Limit Enforcement: The 201st request from a single IP address within a 15-minute window returned HTTP 429 Too Many Requests, confirming correct rate limit enforcement.
- CORS Rejection: API requests originating from unlisted origins received a CORS error response, confirming correct CORS whitelist enforcement.
- Payment Tampering: Submissions of payment verification requests with incorrect HMAC signatures consistently resulted in PAYMENT_FAILED order status, confirming that payment fabrication is not possible.
- Token Reuse Detection: Submission of a previously invalidated refresh token correctly returned HTTP 401, confirming the token rotation and reuse detection mechanism.

9.5 Test Results Summary

The comprehensive testing programme yielded the following outcomes:

- All 28 defined functional requirements were verified and confirmed to be correctly implemented.
- The order state machine was validated to correctly enforce all permitted status transitions and reject all invalid transitions.
- All 15 security test scenarios were passed, confirming effective defence against common web application vulnerabilities.
- Performance testing confirmed acceptable response times under normal and medium load conditions, with clear horizontal scaling recommendations for high-volume deployment.
- All 15 documented test cases passed their expected results without modification to the codebase.

Chapter 10

CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

HomeBite – Home-Made Food Marketplace Platform addresses a genuine, well-defined, and commercially significant gap in the food technology ecosystem. The home-cooked food segment, which encompasses millions of talented home cooks and a large, underserved consumer base, has until now operated exclusively through informal, non-scalable, and trust-deficient channels. Large commercial food delivery aggregators have not — and structurally cannot — serve this segment due to their requirement for formal commercial restaurant licensing. HomeBite fills this gap with a purpose-built, production-grade digital marketplace specifically designed for home cooks.

This project has successfully demonstrated the design and implementation of a comprehensive, full-stack marketplace platform that addresses the problem across its complete operational scope: from vendor discovery and verified onboarding, through menu management, cart and ordering, secure payment processing, logistics integration, and post-delivery reviews. The platform is not a prototype or a proof-of-concept, but rather a production-ready system built on modern, industry-standard technologies and architectural patterns.

The key technical contributions of this project are:

22. A modular, layered REST API architecture in Node.js and Express.js that cleanly separates routing, request validation, business logic, and data access concerns, enabling independent evolution and testing of each domain module.
23. A JWT dual-token authentication system with rotation and reuse detection that provides both security through short-lived access tokens and usability through long-lived silent refresh tokens, without requiring server-side session storage.
24. A Prisma-managed relational database schema for MySQL 8.0 with proper foreign key constraints, enumerated types, soft deletes, and price snapshotting to maintain historical data integrity across all transactional records.

25. A cross-platform React Native mobile application built with Expo SDK 51, featuring role-based navigation through Expo Router, automatic access token refresh through Axios interceptors, persistent cart management through Zustand, and encrypted native token storage through expo-secure-store.
26. Stubbed integrations with Razorpay and Shiprocket that implement the full API contract of the live services, including HMAC-SHA256 payment signature verification, enabling seamless transition to production with credential configuration alone.
27. A comprehensive, defence-in-depth security posture incorporating bcrypt password hashing, JWT RBAC, Joi input validation, Helmet HTTP headers, CORS whitelisting, and IP-based rate limiting.
28. A containerised, CI/CD-enabled deployment architecture using Docker, Docker Compose, and GitHub Actions that supports horizontal scaling and automated, zero-downtime deployments.

The platform has been comprehensively tested and validated against all 28 functional requirements, all security requirements, and established performance benchmarks. Comprehensive testing confirmed that HomeBite correctly enforces the order state machine, maintains payment integrity, and provides role-appropriate access control across all API endpoints.

In summary, HomeBite demonstrates that a production-grade two-sided marketplace — covering authentication, multi-role access control, inventory management, payment processing, logistics integration, and a cross-platform mobile frontend — can be designed and implemented as a cohesive, well-architected system using modern open-source technologies within an academic project timeframe. The platform represents a meaningful step toward the digital formalisation of the informal home-cooked food economy, with the potential to create real economic opportunity for home cooks and genuine value for consumers.

10.2 Future Scope

While the current version of HomeBite fulfils all primary functional and non-functional objectives, the architecture has been deliberately designed for extensibility. The following enhancements are planned for future development iterations:

AI-Powered Personalised Recommendations

The order history data accumulated by HomeBite provides a rich foundation for collaborative filtering and content-based recommendation algorithms. Future versions will incorporate an AI recommendation engine that analyses customer order history, time-of-day ordering patterns, dietary preferences, and geographic proximity to suggest relevant vendors and food items. This feature will be implemented as a separate microservice that consumes the HomeBite API and provides personalised recommendation feeds.

Location-Based Vendor Discovery with Map View

The Vendor and Address models in the current schema already store latitude and longitude coordinates, providing the data foundation for geospatial features. Future development will implement proximity-based vendor filtering using the Haversine formula, enabling customers to discover vendors within a specified radius. A visual map view using the react-native-maps library will allow customers to browse nearby vendors spatially, significantly improving the discovery experience.

Real-Time Order Tracking and Push Notifications

Future versions will integrate Expo Push Notifications and Firebase Cloud Messaging (FCM) to deliver real-time push notifications to customers and vendors on order status changes. WebSocket or Server-Sent Events (SSE) connectivity will provide live order tracking without polling, enabling customers to see live preparation and delivery status updates within the application.

Vendor Payout Automation

The Razorpay Payout API will be integrated in a future version to automate periodic vendor payout disbursements based on completed and settled orders. Vendors will have access to a dedicated earnings dashboard displaying settlement history, pending payouts, revenue trends, and order analytics to support informed business decision-making.

OTP-Based Phone Verification

The OtpToken model is already defined in the current Prisma schema, providing the data layer for this feature. Integration with an SMS gateway provider (MSG91 or Twilio) will enable OTP-based phone number verification during vendor onboarding, adding an additional layer of identity verification and increasing consumer trust in verified vendors.

Discount and Coupon System

A flexible discount and coupon system will be implemented to support both vendor-specific promotional codes and platform-wide discount campaigns. The system will support percentage discounts, flat-rate discounts, minimum order value conditions, single-use coupons, and expiry dates, enabling both marketing campaigns and customer retention initiatives.

Web-Based Admin Dashboard

A React.js web application with a comprehensive admin interface will be developed to complement the mobile application. The web dashboard will feature real-time platform analytics with interactive charts, a vendor approval queue with FSSAI document preview, an order dispute resolution interface, and bulk user management capabilities, providing administrators with a more ergonomic and powerful platform management experience.

Multi-Language and Localisation Support

To support HomeBite's expansion to regional markets across India with diverse linguistic communities, future versions will implement React Native's i18next-based localisation framework supporting Hindi and regional languages including Telugu, Tamil, Marathi, and Kannada, ensuring the platform is accessible to a wider audience of home cooks and customers.

These planned enhancements will progressively expand HomeBite from its current comprehensive MVP into a fully featured, AI-augmented, globally scalable marketplace platform. The modular architecture and clean separation of concerns deliberately implemented in the current version ensure that each enhancement can be developed and deployed independently without disrupting the existing system.

REFERENCES

[1] Node.js Foundation. (2023). Node.js – JavaScript runtime built on Chrome's V8 engine. Retrieved from https://nodejs.org/

[2] Express.js. (2023). Fast, unopinionated, minimalist web framework for Node.js. Retrieved from https://expressjs.com/

[3] Prisma. (2024). Prisma – Next-generation Node.js and TypeScript ORM. Prisma Documentation. Retrieved from https://www.prisma.io/docs

[4] Oracle Corporation. (2023). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/

[5] Meta Platforms, Inc. (2023). React Native Documentation. Retrieved from https://reactnative.dev/

[6] Expo. (2024). Expo SDK 51 Documentation. Retrieved from https://docs.expo.dev/

[7] Razorpay. (2024). Razorpay Payment Gateway API Documentation. Retrieved from https://razorpay.com/docs/

[8] Shiprocket. (2024). Shiprocket API Documentation. Retrieved from https://apidocs.shiprocket.in/

[9] Rochet, J-C., and Tirole, J. (2003). Platform Competition in Two-Sided Markets. Journal of the European Economic Association, 1(4), 990–1029. doi:10.1162/154247603322493212

[10] Pavlou, P. A., and Fygenson, M. (2006). Understanding and Predicting Electronic Commerce Adoption: An Extension of the Theory of Planned Behavior. MIS Quarterly, 30(1), 115–143.

[11] Yeh, H., Li, S., and Mahatanankoon, P. (2020). Mobile Commerce Adoption Factors: A Consumer Perspective. International Journal of Electronic Commerce, 24(3), 1–29.

[12] Newman, S. (2015). Building Microservices: Designing Fine-Grained Systems (1st Edition). O'Reilly Media.

[13] Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning Publications.

[14] Auth0. (2023). JWT (JSON Web Token) Introduction. Retrieved from https://jwt.io/introduction

[15] OWASP Foundation. (2023). OWASP Top 10 – Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/

[16] Pressman, R. S. (2019). Software Engineering: A Practitioner's Approach (9th Edition). McGraw Hill Education.

[17] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd Edition). Addison-Wesley Professional.

[18] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.